

SRTF Documentation

1-Correct Process Selection

- Choose process with **Shortest Remaining Time**. “Remaining Time = Burst Time – Executed Time”

c	Arrival	Burst
P1	0	7
P2	2	4

When $t = 2$,

Remaining P1 = 5

Burst P2 = 4

Then we will choose P2

Notes:

- Check **new Arrivals** always.
- It is same as SJF (Shortest Job First) but SJF is non preemptive.
- It is a preemption Algorithm so it can stop processing and choose another One.

2-Arrival Handling

- Correct Dealing with new coming processes.
Each time new coming process -> compare it with current Process

3-Tie Handling “equality case”

- If burst for more than one process is equal or we can say remaining time for more than one process is equal in this case, we call it Tie case Then we will follow these rules:
 - Tie in Remaining Time
- Occurs when: Two processes have the same remaining time
- How to solve: Ties are resolved using FCFS “First come first serve”.

The one that will arrive first and have same remaining time will serve first

SRTF Documentation

P	Arrival	Burst
P1	0	5
P2	1	5

➔ Then in this case we will start P1 until it ends then P2.

2-Tie in Remaining Time AND Arrival Time

- Occurs when: Two processes have the same remaining times and arrival times
- How to solve: Ties are resolved using Process ID order rule "e.g.: P1 has smaller ID than P2 so P1 before P2".

P	Arrival	Burst
P1	0	5
P2	0	5

At $t = 0$:

Remaing P1 = 5 And Remaining P2 = 5 (Double tie)

➔ Then we follow process ID order then P1 will run until end then P2.

3-Tie after preemption

- Occurs when: when new process arrives but its burst equal remaining time
- How to solve: Ties are resolved using No preemption. "Continue without stopping the process"

P	Arrival	Burst
P1	0	6
P2	2	4

SRTF Documentation

At $t = 2$

Remaining $P1 = 4$ And burst $P2 = 4$.

➔ Then continue $P1$ without stopping "No preemption".

4-Multiple Process Tie

- Occurs when: Many processes have same remaining time.
- How to solve: Ties are resolved using
 1. Arrival time
 2. Process ID

P	Arrival	Burst
P1	0	4
P2	1	3
P3	2	3

➔ $P1 \rightarrow P2 \rightarrow P3$

Recommended Tie-Breaking Rules:

-In summary, let's sum it up.

Use this **order**:

1. Shortest Remaining Time
2. Earliest Arrival Time
3. Smallest Process ID

4-Completion Behavior

SRTF Documentation

➔ When process ends, we choose next process with shortest remaining time but it in same time it arrived yet “in Queue”.

5-Completion Behavior

- Preemption occurs when:

$Burst_new < Remaining_current$

- Always check arrivals
 - Update remaining time
-

SRTF algorithm Cases:

Case1: No preemption

P	Arrival	Burst
P1	0	3
P2	4	5

- No process arrives via running time.
- Process arrives via running time, but with higher remaining time.

Case2: With preemption

P	Arrival	Burst
P1	0	8
P2	1	4

- Process arrives via running time with burst/remaining time less than running one.

SRTF Documentation

Case3: Multiple preemption

P	Arrival	Burst
P1	0	8
P2	1	4
P3	2	2

P1 -> P2 -> P3 -> P2 -> P1

- Process arrives via running time with burst/remaining time less than running one. "When it happens, many times"

Case4: Tie case

P	Arrival	Burst
P1	0	5
P2	1	5

Here we will follow FCFS.

- When equality case happens for two processes or more then we should define the rule

SRTF Documentation

Case5: Idle Time case

P	Arrival	Burst
P1	2	4

CPU is idle through the period from 0: 2.

- **When CPU doesn't run any process then it is idle.**

Case6: conflict Case "priority vs SRTF"

P	Priority	Arrival	Burst
P1	1	0	8
P2	5	1	4

Here Priority chooses P1 to run and SRTF chooses P2.

- **When priority algorithm chooses process and SRTF chooses another process.**
-

Before pseudocode algorithm, rules that we will follow:

Tie-breaking rule:

1. Shortest Remaining Time
2. Earliest Arrival Time
3. Smallest Process ID

SRTF Documentation

Pseudocode For SRTF:

/*n -> number of processes, arrival [i] -> arrive time of process i, burst [i] -> burst time of process i*/

```
Algorithm SRTF(n, arrival, burst){
    remaining ← Array of size n;
    waiting_time ← Array of size n (Initialized to 0);
    turnaround_time ← Array of size n (Initialized to 0);
    for i:=0 to n-1 step 1 do{
        remaining[i] ← burst[i];
    }
    complete ← 0;
    time ← 0;
    shortest ← -1;
    min_remaining ← Infinity
    while complete < n do {
        min_remaining ← Infinity;
        shortest ← -1;
        for i :=0 to n-1 step 1 do {
            if arrival[i] <= time AND remaining[i] > 0 then {
                if arrival[i] < min_remaining then {
                    min_remaining ← remaining[i];
                    shortest ← i;
                }
            }
        }
        if shortest > -1 then {
            time ← time + remaining[shortest];
            remaining[shortest] ← remaining[shortest] - 1;
            if remaining[shortest] == 0 then complete ← complete + 1;
        }
    }
}
```

SRTF Documentation

```
    }
    else if remaining[i] = min_remaining[i] then {
        if arrival[i] < arrival[shortest] then {
            shortest  $\leftarrow$  i;
        } else if arrival[i] = arrival[shortest] then {
            if i < shortest then {
                shortest  $\leftarrow$  i;
            }
        }
    }
}

if shortest = -1 then {
    write ("Idle");
    current_time++;
} else {
    if start_time[shortest] = -1 then {
        start_time[shortest]  $\leftarrow$  current_time;
        RT[shortest  $\leftarrow$  start_time[shortest] - arrival [shortest];
    }
    remaining[shortest]--;
    current_time++;
    if remaining[shortest] = 0 then {
        complete ++;
```


SRTF Documentation

```
finish_time  $\leftarrow$  time + 1;  
waiting_time  $\leftarrow$  finish_time - burst[shortest] - arrival[shortest];  
turnaround_time  $\leftarrow$  finish_time - arrival[shortest];  
time  $\leftarrow$  time + 1;  
}  
}  
}
```